DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Shipping Itch.io

CMSC 197 GDD Shipping Guide

Prepared by: Rene Andre B. Jocsing

Published: March 10, 2026

Guide Objectives

- Install and configure Godot export templates for Web, Desktop, and Android targets
- Understand the HTML5 export requirements for in-browser play
- Create and configure an Itch.io project page correctly
- Upload builds manually and via the `butler` CLI tool
- Automate future releases with a repeatable deploy workflow

Overview

Itch.io is an open marketplace for independent games and creative software (link). It is the standard target for this course because it requires zero upfront cost, supports in-browser HTML5 play, and allows creators to set their own pricing and access conditions. Your projects may be published here.

In real game development, publishing is not optional polish. **A game that cannot be shipped is not finished.** Export and deployment are engineering tasks that surface real bugs—missing resources, wrong renderer settings, broken paths—that simply do not appear during in-editor testing.

Part 1: Preparing Your Godot Project

Before touching the export dialog, your project must be in a shippable state.

Project Settings Checklist

Project → **Project Settings** → **General**:

- **Application / Config / Name:** Set a real name, not `New Game`
- **Application / Config / Version:** Use semantic versioning, e.g. `1.0.0`
- **Display / Window / Viewport Width / Height:** Confirm these match your game's intended resolution
- **Display / Window / Stretch / Mode:** Set to `canvas_items` (recommended for 2D games) (link)
- **Display / Window / Stretch / Aspect:** Set to `keep` or `keep_width` depending on your layout

Resolution Convention: 1280×720

Use 1280×720 as your base viewport for Itch.io web embeds. The default embed canvas is 960 pixels wide; a 16:9 base scales cleanly. Avoid non-standard aspect ratios unless the design demands it—they produce letterboxing or pillarboxing that looks unpolished.

Resource Import Hygiene

Godot exports only files that are either listed in the project or reachable from the main scene tree. Common omissions:

- Audio files referenced by path string rather than `preload()`
- Fonts assigned via the Inspector but not imported
- Shader files in subdirectories not scanned at startup

Run **Project** → **Export** once and review the console output for missing resource warnings *before* uploading.

Common Pitfalls

Path-string resource loading is fragile:

- `load("res://sfx/jump.ogg")` will silently return `null` if the file is missing at runtime
- Use `preload()` for critical assets so errors surface at compile time
- All `res://` paths are case-sensitive on Linux and on the Itch.io server; `Jump.ogg` ≠ `jump.ogg`

Autoload Singletons

If your project uses Autoload singletons, verify them in **Project** → **Project Settings** → **Autoload**. Any singleton that is listed but whose file has been moved or renamed will cause an export failure.

```gdscript
# game_manager.gd
extends Node

# Typed members prevent silent null dereferences
var score: int = 0
var current_level: int = 1
var _is_playing: bool = false

func reset() -> void:
    score = 0
    current_level = 1
    _is_playing = false
```

Part 2: Installing Export Templates

Godot separates the editor from the export templates. You must install templates before any export target is available (link).

Installation Steps

1. Open **Editor** → **Manage Export Templates**
2. Click **Download and Install** for the version matching your editor
3. Wait for the download to complete (roughly 300–500 MB)
4. Confirm the status reads *Templates Installed*

Template Versions Must Match

A Godot 4.3 editor requires Godot 4.3 templates. Mismatched versions produce silent export failures or corrupted binaries. If your editor auto-updated, reinstall templates.

If you are on a slow connection, download templates manually from godotengine.org/download/archive and install via **Install from File**.

Part 3: Configuring Export Presets

Open **Project** → **Export**. You will create one preset per target platform.

Web (HTML5) Export

This is the most convenient target for Itch.io builds. Players launch it directly in their browser with no installation required.

Steps:

1. Click **Add** → **Web**
2. Name it `Web`
3. Under **Options**:
   - **Vram Texture Compression / For Desktop:** On
   - **Vram Texture Compression / For Mobile:** On
   - **Export Type:** `GDExtension` (leave default unless using C# or native plugins)
   - **Progressive Web App:** Off (but try it if it works for your project)
4. Under **Custom Template:** Leave blank (uses installed templates)

Export the files:

1. Click **Export Project** (not *Export PCK/Zip*)
2. Create a folder: `export/web/`
3. Set filename: `index.html`
4. Click **Save**

Godot will generate: `index.html`, `index.js`, `index.wasm`, `index.pck`, and `index.png`.

Why `index.html`?

Itch.io's HTML5 host looks for `index.html` as the entry point for web games. Naming it anything else requires manual configuration in the Itch.io embed settings. Save yourself the confusion (link).

Windows Export

1. Click **Add** → **Windows Desktop**
2. Name it `Windows`
3. Under **Options / Binary Format:** Choose `64-bit` (covers all modern Windows installs)
4. Click **Export Project**, output to `export/windows/`, filename `GameName.exe`

Zip the output folder before uploading to Itch.io. Itch.io expects a `.zip` for desktop builds and will unpack it on the server.

Linux Export

1. Click **Add** → **Linux**
2. Name it `Linux`
3. Click **Export Project**, output to `export/linux/`, filename `GameName.x86_64`
4. Zip the output folder

Itch.io Note

macOS requires a paid Apple Developer account to distribute outside the App Store without triggering Gatekeeper warnings. For this course, Web + Windows is sufficient (link).

Part 4: The SharedArrayBuffer Problem

This is the single most common blocker for Godot HTML5 games on Itch.io. Read this carefully.

What Is It?

Godot's web export uses WebAssembly threads for performance. Threads require `SharedArrayBuffer`, which browsers block unless the page is served with specific HTTP headers (link):

```
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
```

Itch.io does *not* set these headers on all game pages by default.

Solution A: Enable SharedArrayBuffer in Itch.io (Preferred)

1. After uploading your build, go to the game's **Edit Game** page
2. Scroll to **Embed Options**
3. Check **SharedArrayBuffer support** (available to creators who request it)
4. Email `support@itch.io` requesting the feature if not visible (link)

Solution B: Disable Threads in Godot

If SharedArrayBuffer is unavailable, disable threading in the export:

1. **Project** → **Export** → Web preset
2. **Options** → **Threads Support:** Off
3. Re-export and re-upload

This reduces performance on complex scenes but is perfectly acceptable for 2D games in this course.

Common Pitfalls

Symptom vs. cause:

- Symptom: Black screen or *"Your browser does not support…"* error
- Cause: Missing COOP/COEP headers (SharedArrayBuffer blocked)
- Symptom: Game loads but runs at 5–10 FPS
- Cause: Threads disabled but you are rendering a 3D scene

Do not guess. Open the browser DevTools console (**F12**) and read the actual error message before changing settings.

Part 5: Creating Your Itch.io Page

Account Setup

1. Register at itch.io/register
2. Under **Account** → **Settings**: Set a display name and profile picture
3. Under **Account** → **Settings** → **Creator Settings**: Enable *Enable developer tools*

Creating the Game Page

Navigate to **Dashboard** → **Create new project**.

Required Fields:

- **Title:** Your game's name (not `cmsc197-mp1`)

- **Kind of project:** *HTML5* for web builds, *Downloadable* for desktop-only
- **Classification:** *Games*
- **Short description:** One sentence. Write it like a game pitch, not a homework description.
- **Cover image:** 315×250 px minimum (link). This is mandatory for discovery.

Recommended Fields:

- **Tags:** At least 3 genre/mechanic tags (e.g., `2d`, `godot`, `arcade`)
- **Description:** Explain controls, objective, and any known issues
- **Screenshots:** At least 3 in-game screenshots
- **Genre:** Helps surface your game in search

Page Quality Is Part of the Grade

A game page with a blank description, no cover image, and no tags reflects the same lack of care as unformatted code. Your game page is the first thing a player sees. Treat it with the same rigor you apply to your scene architecture.

Uploading the Web Build

1. In **Uploads**, click **Upload files**
2. Upload a `.zip` containing all files from your `export/web/` folder
3. Check **This file will be played in the browser**
4. Set **Embed dimensions** to match your viewport (e.g., 1280×720)
5. Click **Save & view page**

Test the embedded game immediately. Do not submit a link to a page you have not personally played in a browser.

Uploading Desktop Builds

1. Upload the Windows `.zip`, check **Windows**
2. Upload the Linux `.zip`, check **Linux**
3. Set pricing to *No payments* or *Pay what you want* at $0 minimum

Itch.io Note

Set your game's visibility to **Restricted** while developing, and switch to **Public** only when ready. A broken public page cannot be unindexed from search immediately (link).

Part 6: Butler — Automated Deployment

Manually uploading a `.zip` for every build is error-prone. Butler is Itch.io's official command-line deployment tool (link). It performs delta uploads (only changed files), version-tracks builds, and integrates cleanly into shell scripts.

Installation

```
# Download the binary
curl -L -o butler.zip \
  https://broth.itch.ovh/butler/linux-amd64/LATEST/archive/default
unzip butler.zip
chmod +x butler

# Move to your PATH
sudo mv butler /usr/local/bin/
```

On Windows, download `butler.exe` from itchio.itch.io/butler and add it to your `PATH`.

Authentication

```
butler login
# Opens a browser window; authorize the app
# Credentials are stored in ~/.config/itch/butler_creds
```

Pushing a Build

```
# Format: butler push <local_folder> <username>/<game-slug>:<channel>
butler push export/web/ yourusername/your-game-slug:html5

# Push a Windows build from a zip
butler push export/windows/ yourusername/your-game-slug:windows

# Push a Linux build
butler push export/linux/ yourusername/your-game-slug:linux
```

Channel names are arbitrary labels you choose. Use descriptive names: `html5`, `windows`, `linux`. Each channel maintains its own version history on Itch.io (link).

Delta Uploads Save Time

Butler computes a binary diff between the previous and new build. For a 20 MB game, a typical code-only patch might upload less than 1 MB. This matters when iterating rapidly before a deadline.

Deployment Script

Wrap the full pipeline in a script so you never forget a step:

```
#!/usr/bin/env bash
set -e  # Exit on any error

GODOT="/path/to/godot"
PROJECT_PATH="."
GAME_SLUG="yourusername/your-game-slug"

# Clean previous exports
rm -rf export/
mkdir -p export/web export/windows export/linux

# Export all presets
"$GODOT" --headless --export-release "Web" \
    export/web/index.html
"$GODOT" --headless --export-release "Windows" \
    export/windows/GameName.exe
"$GODOT" --headless --export-release "Linux" \
    export/linux/GameName.x86_64

# Push to Itch.io
butler push export/web/     "$GAME_SLUG:html5"
butler push export/windows/ "$GAME_SLUG:windows"
butler push export/linux/   "$GAME_SLUG:linux"

echo "Deployment complete."
```

Common Pitfalls

Headless export requires the project to compile cleanly:

- GDScript parse errors abort the export silently; check the exit code (`echo $?`)
- The `--export-release` flag requires a preset with that exact name in `export_presets.cfg`
- `export_presets.cfg` must be committed to version control for CI pipelines to work

---

Part 7: Android Builds

Android is the only mobile target that does not require a paid developer account to distribute outside an official store. Itch.io hosts the `.apk` as a standard downloadable, and players sideload it directly. This makes it the correct mobile target for course projects (link).

APK vs. AAB

Google Play requires the Android App Bundle (`.aab`) format. Itch.io does not use the Play Store. Always export an `.apk` for Itch.io distribution—`.aab` files cannot be sideloaded directly by players.

Prerequisites

Android export requires three pieces of external tooling that Godot does not bundle.

1. Java Development Kit (JDK 17)

```
sudo apt install openjdk-17-jdk
java -version  # Confirm: openjdk 17.x.x
```

On Windows, download the JDK 17 installer from adoptium.net and add `JAVA_HOME` to your environment variables.

2. Android SDK

1. Install **Android Studio** from developer.android.com/studio (link)
2. Open **SDK Manager** → **SDK Platforms**: install **Android 8.0 (API 26)** or higher
3. Open **SDK Manager** → **SDK Tools**: install **Android SDK Build-Tools** and **Command-line Tools**
4. Note the SDK path (e.g., `~/Android/Sdk` on Linux)

You do not need to use Android Studio as an IDE. It is only required here for its SDK manager.

3. Godot Editor Settings

1. **Editor** → **Editor Settings** → **Export** → **Android**
2. Set **Android SDK Path** to your SDK location
3. Set **Java SDK Path** to your JDK 17 installation
4. Godot will display a green checkmark next to each field when the paths are valid

Common Pitfalls

JDK version must be 17, not 21 or later. Godot 4's Gradle build scripts are not yet compatible with JDK 21 as of the 4.3 stable release. Using the wrong JDK produces cryptic Gradle errors that have nothing to do with your game code (link).

Creating a Keystore

Android requires all `.apk` files to be digitally signed, even for sideloading. You will generate a self-signed *debug keystore* for development and Itch.io distribution.

```
keytool -genkey -v \
  -keystore debug.keystore \
  -storepass android \
  -alias androiddebugkey \
  -keypass android \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000 \
  -dname "CN=Android Debug,O=Android,C=US"
```

Store `debug.keystore` somewhere permanent—outside your project folder. **Do not commit it to version control.** Add it to your `.gitignore`.

Debug vs. Release Keystore

For Itch.io distribution, a debug keystore is sufficient. A release keystore is only required for Google Play. The passwords above (`android`/`android`) are the standard Android debug keystore defaults and are publicly known—do not reuse them for a release build.

Configuring the Android Export Preset

1. **Project** → **Export** → **Add** → **Android**
2. Name the preset `Android`

Required fields under Options:

- **Package / Unique Name:** Reverse-domain format, e.g. `io.itch.yourusername.gameslug`. Must be globally unique.
- **Package / Version Code:** An integer incremented with every release (start at `1`)
- **Package / Version Name:** Human-readable version string, e.g. `1.0.0`
- **Keystore / Debug / Keystore:** Path to `debug.keystore`
- **Keystore / Debug / User:** `androiddebugkey`
- **Keystore / Debug / Password:** `android`

Recommended fields:

- **Screen / Orientation:** Set to `landscape` or `portrait` to lock rotation
- **Graphics / Opengl Debug:** Off for release builds
- **Permissions:** Only enable what your game actually uses. Requesting unused permissions will flag your APK as suspicious.

Itch.io Note

Internet permission is off by default. If your game makes HTTP requests (e.g., leaderboards, analytics), you must explicitly enable **Permissions** → **Internet** in the preset. Without it, all network calls silently fail on device.

Exporting the APK

1. Click **Export Project** in the Export dialog
2. Create folder `export/android/`
3. Set filename: `GameName.apk`
4. Click **Save**

Godot invokes Gradle internally; the first build takes 2–5 minutes as dependencies are downloaded. Subsequent builds are faster due to Gradle's build cache.

Common Pitfalls

Common Android export errors and their causes:

- *SDK not found*: SDK path in Editor Settings is wrong or SDK Tools are not installed
- *Failed to find Build-Tools*: Open Android Studio SDK Manager, install Build-Tools
- *Keystore file not found*: The keystore path is a relative path that breaks when Godot changes working directory; use an absolute path
- *INSTALL_FAILED_UPDATE_INCOMPATIBLE*: You are reinstalling over a build signed with a different keystore; uninstall the old APK from the device first

Testing on Device Before Upload

Do not upload to Itch.io without testing on a real device or emulator first.

Physical device (preferred):

1. Enable **Developer Options** on the Android device (tap *Build Number* seven times in *About Phone*)
2. Enable **USB Debugging**
3. Connect via USB; run `adb devices` to confirm it is detected
4. Run `adb install export/android/GameName.apk`

Emulator (fallback):

1. Open **Android Studio** → **Device Manager** → **Create Virtual Device**
2. Choose a Pixel device profile and a recent API image (API 30+)
3. Start the emulator, then run `adb install export/android/GameName.apk`

Test on the Lowest Hardware You Can Find

Emulators run on your workstation's GPU and dramatically over-represent real-device performance. If you have access to a budget Android phone, test on that. A game that runs at 60 FPS in the emulator may drop to 30 on an entry-level device.

GDScript: Touch Input

Keyboard and mouse input does not exist on Android. You must handle touch input explicitly.

```
extends CharacterBody2D

const SPEED: float = 200.0

var _touch_direction: Vector2 = Vector2.ZERO


func _input(event: InputEvent) -> void:
    # Keyboard/gamepad via InputMap actions (desktop and Android with
    # a physical controller)
    pass


func _process(delta: float) -> void:
    var direction: Vector2 = Vector2.ZERO

    # Desktop: read InputMap actions
    direction.x = Input.get_axis("ui_left", "ui_right")
    direction.y = Input.get_axis("ui_up", "ui_down")

    # Mobile: blend with touch direction if no hardware input
    if direction == Vector2.ZERO:
```

```
        direction = _touch_direction

    velocity = direction.normalized() * SPEED
    move_and_slide()


func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventScreenTouch:
        if event.pressed:
            # Convert screen position to direction from screen centre
            var centre := get_viewport_rect().size / 2.0
            _touch_direction = (event.position - centre).normalized()
        else:
            _touch_direction = Vector2.ZERO
```

Design Pattern: Screen-Centre Joystick

For simple 2D games, computing direction relative to screen centre is the quickest touch control to implement. For anything requiring precision, add an on-screen virtual joystick using a `TouchScreenButton` or a dedicated plugin (link).

Screen Size and Safe Areas

Android devices vary enormously in resolution and aspect ratio. Notches and navigation bars eat into the usable area.

```
extends Node

func _ready() -> void:
    var safe_area: Rect2i = DisplayServer.get_display_safe_area()
    var screen_size: Vector2i = DisplayServer.screen_get_size()

    # Log both so you can debug layout issues on device
    if OS.is_debug_build():
        print("Screen: ", screen_size)
        print("Safe area: ", safe_area)
```

For UI-heavy games, wrap your `CanvasLayer` root in a `MarginContainer` and set its margins dynamically from `get_display_safe_area()` to keep buttons away from notches and rounded corners.

Uploading to Itch.io and Butler

Android APKs are uploaded as downloadable files, not in-browser plays. On the Itch.io edit page, upload the `.apk` and check **Android** in the platform selector.

Via butler:

```
butler push export/android/ yourusername/your-game-slug:android
```

Updated deploy.sh with Android target:

```
#!/usr/bin/env bash
set -e

GODOT="/path/to/godot"
```

```
GAME_SLUG="yourusername/your-game-slug"

rm -rf export/
mkdir -p export/web export/windows export/linux export/android

"$GODOT" --headless --export-release "Web" \
    export/web/index.html
"$GODOT" --headless --export-release "Windows" \
    export/windows/GameName.exe
"$GODOT" --headless --export-release "Linux" \
    export/linux/GameName.x86_64
"$GODOT" --headless --export-release "Android" \
    export/android/GameName.apk

butler push export/web/     "$GAME_SLUG:html5"
butler push export/windows/ "$GAME_SLUG:windows"
butler push export/linux/   "$GAME_SLUG:linux"
butler push export/android/ "$GAME_SLUG:android"

echo "Deployment complete."
```

Itch.io Note

Headless Android export still invokes Gradle under the hood. The first headless build on a fresh machine may take 10+ minutes downloading Gradle dependencies. Subsequent builds are fast. Run at least one manual export from the editor first to pre-warm the Gradle cache.

Part 8: GDScript Considerations for Export

Certain GDScript patterns behave differently in exported builds.

Debug vs. Release Mode

```
extends Node

func _ready() -> void:
    if OS.is_debug_build():
        # Only runs in editor and debug exports
        print("Debug mode active")
        _show_hitboxes()


func _show_hitboxes() -> void:
    # Reveal collision shapes for testing
    get_tree().call_group("debug_visuals", "show")
```

Never Rely on `print()` in Production

`print()` output is visible in the browser console (F12 → Console). Players will find it. Use `OS.is_debug_build()` to gate all diagnostic output. This is also good hygiene for the desktop builds.

File Paths in Exported Builds

```
# CORRECT: res:// paths work in both editor and export
var texture := preload("res://assets/player.png")
```

```

# CORRECT: user:// paths for save data (writable at runtime)
const SAVE_PATH: String = "user://save_data.cfg"

func save_game(data: Dictionary) -> void:
    var config := ConfigFile.new()
    for key: String in data.keys():
        config.set_value("save", key, data[key])
    config.save(SAVE_PATH)


# WRONG: Absolute or relative OS paths break in export
# var bad := load("/home/user/game/assets/player.png")  # Never do this
```

`@export` ***Variables and Scene Wiring***

```
extends Node

@export var coin_scene: PackedScene
@export var background_music: AudioStream

func _ready() -> void:
    # Guard against missing Inspector assignments
    assert(coin_scene != null, "coin_scene is not assigned in Inspector")
    assert(background_music != null, "background_music is not assigned")
```

Common Pitfalls

`@export` variables that are not assigned in the Inspector are `null` at runtime.

- The editor does not warn you about unassigned exports
- The error surfaces at runtime as *Invalid call. Nonexistent function on base Nil*
- Use `assert()` in `_ready()` to catch this during testing, not during grading

---

Part 9: Final Checklist

Before sharing your Itch.io link:

Android Build Quality

- APK installs successfully via `adb install` with no errors
- Game launches without crashing on a physical device or emulator
- Touch input is responsive and maps correctly to game actions
- UI elements are within the safe area on notched devices
- Performance is acceptable on a mid-range device (target: stable 30+ FPS)

Build Quality

- Game loads without a black screen or console errors
- All scenes transition correctly (title → gameplay → game over → restart)
- Audio plays (check both SFX and BGM if applicable)
- Input works as expected (keyboard, mouse, or gamepad depending on game)
- Game runs at acceptable framerate in the browser (target: 60 FPS)

Page Quality

- Title is the actual game name, not a file path or placeholder
- Cover image is present and correctly sized
- Short description is one complete sentence
- Controls are documented in the description
- At least 3 tags are applied
- At least 1 screenshot is uploaded
- Visibility is set to **Public** (not Draft or Restricted)

Sharing

- Share the full Itch.io URL (not the editor URL, not the dashboard URL)
- Verify the link opens in an incognito window without logging in

Play Your Own Game

Open an incognito window. Go to your Itch.io page. Play the game from start to game-over at least once. This is the single most effective QA step available to you. If you cannot complete this step, the game is not ready to share.